Le Perceptron

1. Qu'est-ce qu'un Perceptron ? Lien avec le neurone biologique

Le **perceptron**, proposé par Frank Rosenblatt à la fin des années 1950, est le plus simple des réseaux de neurones : **un unique neurone artificiel** qui prend plusieurs entrées et produit une sortie binaire. C'est un **classifieur linéaire** supervisé. Il s'inspire du modèle de neurone formel de McCulloch et Pitts (1943).

Neurone biologique	Perceptron
Dendrites (reçoivent les signaux)	Entrées $x_1, \ldots, x_n$
Synapses (renforcent ou affaiblissent un signal)	Poids $w_1, \ldots, w_n$
Corps cellulaire (intègre les signaux)	Somme pondérée $\Sigma\, w_i x_i + b$
Seuil de déclenchement du potentiel d'action	Fonction d'activation (seuil)
Axone (transmet le signal)	Sortie $\hat{y}$

L'analogie est une inspiration, pas une modélisation fidèle : un vrai neurone est bien plus complexe (dynamique temporelle, impulsions, plasticité).

2. Fonction mathématique, usage et termes de l'équation

$$z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b = w \cdot x + b$$
$$\hat{y} = f(z)$$

- $\mathbf{x = (x_1, \ldots, x_n)}$: vecteur des entrées (les caractéristiques de l'exemple).
- $\mathbf{w = (w_1, \ldots, w_n)}$: vecteur des poids. Chaque w_i mesure l'importance et le signe de l'influence de x_i.
- **b** : biais, un seuil réglable qui décale la frontière de décision. Il permet à celle-ci de ne pas passer par l'origine.
- **z** : somme pondérée (potentiel d'activation).
- **f** : fonction d'activation, qui convertit z en sortie.
- $\hat{\mathbf{y}}$: sortie prédite (0 ou 1, ou −1 ou +1).

Usage : la **classification binaire** de données **linéairement séparables**. Géométriquement, l'équation $w \cdot x + b = 0$ définit un **hyperplan** (une droite en 2D) qui sépare l'espace en deux classes.

3. Règles d'apprentissage

Règle du perceptron (Rosenblatt), la principale. Pour chaque exemple (x, y), avec η le taux d'apprentissage (0 < η ⩽ 1) :

```
wᵢ ← wᵢ + η · (y − ŷ) · xᵢ
b  ← b  + η · (y − ŷ)
```

- Si la prédiction est correcte (y = ŷ), rien ne change.
- Si ŷ = 0 alors que y = 1, les poids sont augmentés dans la direction de x.
- Si ŷ = 1 alors que y = 0, ils sont diminués.

Autres règles :

- **Règle de Hebb** : $\Delta w_i = \eta \cdot x_i \cdot y$. Elle renforce les connexions entre unités simultanément actives, d'où l'adage « neurons that fire together, wire together ». Elle est plus ancienne et moins efficace pour la classification.
- **Règle delta (Widrow-Hoff, Adaline)** : $w_i \leftarrow w_i + \eta\,(y - z)\,x_i$. L'erreur est calculée sur la sortie linéaire z, avant activation, ce qui revient à une **descente de gradient** sur l'erreur quadratique. C'est l'ancêtre de la rétropropagation.

4. Fonction d'activation

Le perceptron classique utilise la **fonction échelon de Heaviside** (ou la fonction signe) :

```
f(z) = 1 si z ⩾ 0,  0 sinon
```

Elle produit une sortie binaire, mais elle n'est pas dérivable, ce qui interdit la descente de gradient. D'où le choix d'autres fonctions dans les réseaux modernes :

- **Sigmoïde** : sortie entre 0 et 1, interprétable comme une probabilité. Un neurone sigmoïde équivaut à une régression logistique.
- **Tanh** : sortie entre −1 et 1.
- **ReLU** : f(z) = max(0, z), la plus courante en Deep Learning.

5. Processus d'entraînement

1. **Initialiser** les poids (à 0 ou à de petites valeurs aléatoires) et le biais.
2. **Choisir** un taux d'apprentissage η et un nombre maximal d'époques.
3. **Pour chaque époque**, parcourir les exemples d'entraînement. Pour chacun :
   - calculer z = w·x + b, puis la prédiction ŷ = f(z) ;
   - calculer l'erreur (y − ŷ) ;

    - mettre à jour les poids et le biais avec la règle du perceptron.
4. **S'arrêter** quand une époque se termine sans erreur de classification, ou quand le nombre maximal d'époques est atteint.

Théorème de convergence du perceptron (Novikoff) : si les données sont linéairement séparables, l'algorithme converge vers une solution en un **nombre fini** de mises à jour.

6. Limites du Perceptron

- **Séparabilité linéaire obligatoire** : il ne résout que les problèmes où une droite ou un hyperplan sépare les classes. L'exemple classique est le **XOR**, démontré impossible par Minsky et Papert (1969), ce qui a contribué au premier « hiver de l'IA ».
- **Non-convergence sur données non séparables** : les poids oscillent indéfiniment, sans solution stable.
- **Classification binaire seulement** : il ne gère pas directement le multiclasse (il faut en combiner plusieurs).
- **Sortie sans nuance** : un résultat dur 0 ou 1, sans probabilité ni mesure de confiance.
- **Pas de frontière optimale** : il trouve *une* séparation, pas la meilleure (contrairement aux SVM, qui maximisent la marge).
- **Sensibilité à l'échelle des variables** et à l'ordre des exemples.

Dépassement : empiler plusieurs couches de perceptrons avec des activations non linéaires donne le **perceptron multicouche (MLP)**. Entraîné par **rétropropagation du gradient**, il résout XOR et approxime des fonctions arbitrairement complexes. C'est le point de départ du Deep Learning.